

Your first multi-agent

An agent on its own can be smart, but things get really interesting when multiple agents team up.

BAF allows building **multi-agent systems** where each agent has a specific role, shares information, and works together to get the job done more efficiently.

In this section, we will walk through a **running example** to see how agents interact in action. The system we are looking at is designed to help developers write source code.

Note

This tutorial may be too advanced for BAF beginners. We recommend you understanding the core ideas of agents ([Wiki](#) section) before diving into this example.

This system is composed by 3 agents:

- **Main Agent:** The agent that interacts with the developer, collecting his/her requests. It will store a piece of code and will ask for code modifications to a coder agent (in this example, we only considered adding new functions).
- **Coder Agent:** This agent receives coding requests (sent by the main agent) and leverages a specialized LLM to generate the requested code. Before replying back the generated code, it is sent to a reviewer agent to check for possible errors or bugs. If something was found, the coder agent iterates again to fix the received issues.
- **Code Reviewer Agent:** This agent receives a piece of code and is in charge of finding errors or bugs, leveraging a specialized LLM.

To enable the communication between agents, they must know their WebSocket URLs, which they will use to send/receive messages.

Learn how to send messages between agents here: [communication-between-agents](#)

Example Workflow:

1. A developer imports the code of an application into the database.
2. The developer asks the system to generate a function for processing user authentication.
3. The **Main Agent** assigns the request to the **Coder Agent** to generate the function.
4. The **Coder Agent** produces the code.
5. The **Coder Agent** sends the request to the **Reviewer Agent**
6. The **Reviewer Agent** analyzes the code for possible bugs or syntax errors.
7. The **Reviewer Agent** finds a bug and sends it back to the **Coder Agent**.
8. The **Coder Agent** updates the code fixing the received issue.
9. The **Coder Agent** sends the request to the **Reviewer Agent**
10. The **Reviewer Agent** does not find any issue and replies OK.
11. The **Coder Agent** receives the OK and replies the new code to the **Main Agent**.

12.The **Main Agent** updates the code database and awaits for a new developer query.

The following sections show the code implementation for each agent.

Developer Main Agent

```
# You may need to add your working directory to the Python path. To do so,
uncomment the following lines of code
# import sys
# sys.path.append("/Path/to/directory/agentic-framework") # Replace with your
directory path
import base64
import logging

from baf.core.agent import Agent
from baf.library.transition.events.base_events import ReceiveTextEvent,
ReceiveFileEvent
from baf.core.file import File
from baf.core.session import Session
from baf.exceptions.logger import logger

# Configure the logging module (optional)
logger.setLevel(logging.INFO)

# Create the agent
agent = Agent('main_agent')
# Load agent properties stored in a dedicated file
agent.load_properties('../config.yaml')
# Define the platform your agent will use
websocket_platform = agent.use_websocket_platform(use_ui=True)

# STATES

initial_state = agent.new_state('initial_state', initial=True)
receive_code_state = agent.new_state('receive_code_state')
awaiting_request_state = agent.new_state('awaiting_request_state')
send_request_state = agent.new_state('send_request_state')
final_state = agent.new_state('final_state')

# INTENTS

yes_intent = agent.new_intent('yes_intent', [
    'yes',
])

no_intent = agent.new_intent('bad_intent', [
    'no',
])

# STATES BODIES' DEFINITION + TRANSITIONS

def initial_body(session: Session):
    websocket_platform.reply(session, "Hello, upload your code before
starting.")

def initial_fallback(session: Session):
    websocket_platform.reply(session, "Please, upload a file before starting.")
```

```

initial_state.set_body(initial_body)
initial_state.when_event(ReceiveFileEvent()).go_to(receive_code_state)
initial_state.set_fallback_body(initial_fallback)

def receive_code_body(session: Session):
    if session.event.file:
        # Receiving a code file on the first interaction
        file: File = session.event.file
        code = base64.b64decode(file.base64).decode('utf-8')
        if session.event.file.type == 'text/x-python':
            code = f"```python\n{code}\n```"
        session.set('code', code)
    elif session.get('new_code'):
        # Receiving an updated code after finishing the agent workflow
        session.set('code', session.get('new_code'))
    websocket_platform.reply(session, "Thanks, I stored your code in my
database. This is how it looks like:")
    websocket_platform.reply(session, session.get('code'))

receive_code_state.set_body(receive_code_body)
receive_code_state.go_to(awaiting_request_state)

def awaiting_request_body(session: Session):
    session.delete('new_code')
    websocket_platform.reply(session, "How can I assist you?")

awaiting_request_state.set_body(awaiting_request_body)
awaiting_request_state.when_event(ReceiveTextEvent()) \
    .with_condition(lambda session: session.event.human) \
    .go_to(send_request_state)

def send_request_body(session: Session):
    websocket_platform.reply(session, "Let's see what I can do...")
    session.send_message_to_websocket(
        url='ws://localhost:8011',
        message={
            "request": session.event.message,
            "code": session.get('code')
        }
    )

send_request_state.set_body(send_request_body)
send_request_state.when_event(ReceiveTextEvent()) \
    .with_condition(lambda session: not session.event.human) \
    .go_to(final_state)

def final_body(session: Session):
    new_code: str = session.event.message
    session.set('new_code', new_code)
    websocket_platform.reply(session, "Take a look at the new code:")
    websocket_platform.reply(session, new_code)
    websocket_platform.reply(session, "Do you want to merge the new code?")
    websocket_platform.reply_options(session, ['Yes', 'No'])

final_state.set_body(final_body)

```

```

final_state.when_intent_matched(yes_intent).go_to(receive_code_state)
final_state.when_intent_matched(no_intent).go_to(awaiting_request_state)

# RUN APPLICATION

if __name__ == '__main__':
    agent.run()

```

Coder Agent

```

# You may need to add your working directory to the Python path. To do so,
# uncomment the following lines of code
# import sys
# sys.path.append("/Path/to/directory/agentic-framework") # Replace with your
# directory path
import logging

from baf.core.agent import Agent
from baf.library.transition.events.base_events import ReceiveJSONEvent
from baf.core.session import Session
from baf.exceptions.logger import logger
from baf.nlp.llm.llm_openai_api import LLMOpenAI
from baf.platforms.websocket import WEBSOCKET_PORT

# Configure the logging module (optional)
logger.setLevel(logging.INFO)

# Create the agent
agent = Agent('coder_agent')
# Load agent properties stored in a dedicated file
agent.load_properties('../config.yaml')
agent.set_property(WEBSOCKET_PORT, 8011)
# Define the platform your agent will use
websocket_platform = agent.use_websocket_platform(use_ui=False)

# Create the LLM
gpt = LLMOpenAI(
    agent=agent,
    name='gpt-4o-mini',
    parameters={},
    num_previous_messages=10
)

# STATES

initial_state = agent.new_state('initial_state', initial=True)
generate_code_state = agent.new_state('generate_code_state')
update_code_state = agent.new_state('update_code_state')
reply_code_state = agent.new_state('reply_code_state')

# INTENTS

ok_intent = agent.new_intent('yes_intent', [
    'ok',
])

# STATES BODIES' DEFINITION + TRANSITIONS

```

```

initial_state.when_event(ReceiveJSONEvent()) \
    .with_condition(lambda session: not session.event.human) \
    .go_to(generate_code_state)

def generate_code_body(session: Session):
    message = session.event.message
    new_code: str = gpt.predict(
        message=f"Given the following code:\n\n"
        f"{message['code']}\n\n"
        f"{message['request']}\n\n"
        f"Return only the code (full code with the additions).")
    )
    session.set('new_code', new_code)
    session.send_message_to_websocket(
        url='ws://localhost:8012',
        message=new_code
    )

generate_code_state.set_body(generate_code_body)
generate_code_state.when_intent_matched(ok_intent).go_to(reply_code_state)
# TODO : fix no_intent_matched
generate_code_state.when_no_intent_matched().go_to(update_code_state)

def update_code_body(session: Session):
    issues: str = session.event.message
    new_code: str = gpt.predict(
        message=f"Given the following code:\n\n"
        f"{session.get('new_code')}\n\n"
        f"Update it with the following requirements/fixing these issues\n\n"
        f"(just reply with the new code):\n\n"
        f"{issues}")
    )
    session.set('new_code', new_code)
    session.send_message_to_websocket(
        url='ws://localhost:8012',
        message=new_code
    )

update_code_state.set_body(update_code_body)
update_code_state.when_intent_matched(ok_intent).go_to(reply_code_state)
# TODO : fix no_intent_matched
update_code_state.when_no_intent_matched().go_to(update_code_state)

def reply_code_body(session: Session):
    websocket_platform.reply(session, session.get('new_code'))

reply_code_state.set_body(reply_code_body)
reply_code_state.go_to(initial_state)

# RUN APPLICATION

if __name__ == '__main__':
    agent.run()

```

Code Reviewer Agent

```
# You may need to add your working directory to the Python path. To do so,
uncomment the following lines of code
# import sys
# sys.path.append("/Path/to/directory/agent-framework") # Replace with your
directory path
import logging

from baf.core.agent import Agent
from baf.library.transition.events.base_events import ReceiveTextEvent
from baf.core.session import Session
from baf.exceptions.logger import logger
from baf.nlp.llm.llm_openai_api import LLMOpenAI
from baf.platforms.websocket import WEBSOCKET_PORT

# Configure the logging module (optional)
logger.setLevel(logging.INFO)

# Create the agent
agent = Agent('reviewer_agent')
# Load agent properties stored in a dedicated file
agent.load_properties('../config.yaml')
agent.set_property(WEBSOCKET_PORT, 8012)
# Define the platform your agent will use
websocket_platform = agent.use_websocket_platform(use_ui=False)

# Create the LLM
gpt = LLMOpenAI(
    agent=agent,
    name='gpt-4o-mini',
    parameters={},
    num_previous_messages=10
)

# STATES

initial_state = agent.new_state('initial_state', initial=True)
code_review_state = agent.new_state('code_review_state')

# INTENTS

issues_intent = agent.new_intent('new_function_intent', [
    'issues'
])

ok_intent = agent.new_intent('yes_intent', [
    'ok',
])

# STATES BODIES' DEFINITION + TRANSITIONS

initial_state.when_event(ReceiveTextEvent()) \
    .with_condition(lambda session: not session.event.human) \
    .go_to(code_review_state)

def code_review_body(session: Session):
    code: str = session.event.message
    answer: str = gpt.predict(
        message=f"You are a code reviewer. Given the following code, try to find
if there are syntax errors.\n"
```

```
                f"If you think there are no errors, just reply 'ok'.\n\n"
                f"{code}"
    )
    websocket_platform.reply(session, answer)
```

```
code_review_state.set_body(code_review_body)
code_review_state.go_to(initial_state)
```

```
# RUN APPLICATION
```

```
if __name__ == '__main__':
    agent.run()
```